- Laporan Tugas — ETL Process
  - Daftar Isi
  - 1. Arsitektur & Stack Teknologi
    - Gambaran Sistem
    - Stack Teknologi
    - OLTP Schema (Sumber Data)
  - 2. ERD OLAP Schema
    - Desain: Hybrid Star / Snowflake Schema (Level-1 Snowflake)
    - Penjelasan Tabel
      - Tabel Dimensi (6 tabel)
      - Tabel Fakta (3 tabel)
      - Kalkulasi Measures di fact_sales
  - 3. Step-by-Step: Pembuatan DAG sampai Deploy di Apache Airflow
    - Step 1 — Clone Repository & Persiapan Environment
    - Step 2 — Menjalankan Docker Services
    - Step 3 — Migrasi OLAP Schema (DDL)
    - Step 4 — Implementasi DAG (dag_retail_etl_student.py)
      - Konfigurasi OLAP_DB
      - PHASE 1 — Extract (Sudah Tersedia)
      - PHASE 2 — Load Dimensions
      - PHASE 3 — Load Facts
      - Helper Functions yang Ditambahkan
      - Dependency Graph DAG
    - Step 5 — Deploy ke Apache Airflow
    - Step 6 — Trigger & Monitor DAG di Airflow UI
  - 4. Dokumentasi Run DAG
    - Tampilan Graph DAG di Airflow
    - DAG Runs Summary
    - Status Task per Run (Run Terakhir — Sukses)
  - 5. Query Analitik & Jawaban Pertanyaan Bisnis
    - Bagian A — Desain OLAP Schema & Justifikasi
      - Pilihan Schema: Hybrid Star / Snowflake Schema (Level-1 Snowflake)
    - Pertanyaan 1 — Tren Revenue & Margin per Kategori Produk
      - Query 1a — Revenue & Margin per Kategori per Bulan (Utama)
      - Query 1b — Deteksi Tren Penurunan Margin (Window Function LAG)
    - Pertanyaan 2 — Segmentasi Pelanggan & Metode Pembayaran Favorit
      - Query 2a — Top 5 Pelanggan dengan Metode Pembayaran Favorit (Utama)
      - Query 2b — Distribusi & Korelasi Metode Pembayaran vs Nilai Transaksi
    - Pertanyaan 3 — Efektivitas Kampanye Marketing terhadap Rating Produk
      - Query 3a — Rata-rata Rating per Kategori dengan Status Promo (Utama)
      - Query 3b — Perbandingan Agregat: Kategori Dengan Promo vs Tanpa Promo
      - Query 3c — List Kampanye Beserta Kategori Target dan Avg Rating Produknya
    - Ringkasan Alur Join Query
  - 6. Bonus: Visualisasi Data dengan Metabase

- 📊 Visualisasi 1 — Kategori Paling Profitable
- 📈 Visualisasi 2 — Tren Margin per Kategori per Bulan
- 🏆 Visualisasi 3 — Leaderboard Top 5 Pelanggan
- 🍩 Visualisasi 4 — Distribusi Metode Pembayaran
- ⭐ Visualisasi 5 — Rata-rata Rating per Kategori & Status Promo
- 📋 Visualisasi 6 — Detail Performa Kampanye
- Ringkasan Jawaban Pertanyaan Bisnis
  - P1 — Tren Revenue & Margin
  - P2 — Segmentasi Pelanggan & Metode Pembayaran
  - P3 — Efektivitas Kampanye Marketing
- Lampiran — File Referensi ada di Repository Github

Laporan Tugas — ETL Process

Nama: Titanio Yudista **NIM:** 24120500031 **Prodi:** Sains Data **Mata Kuliah:** Data Warehouse & Data Mining **Github:** Link Github

---

Daftar Isi

1. Arsitektur & Stack Teknologi
2. ERD OLAP Schema
3. Step-by-Step: Pembuatan DAG sampai Deploy di Apache Airflow
4. Dokumentasi Run DAG
5. Query Analitik & Jawaban Pertanyaan Bisnis
   - Bagian A — Desain OLAP Schema & Justifikasi
   - Pertanyaan 1 — Tren Revenue & Margin per Kategori Produk
   - Pertanyaan 2 — Segmentasi Pelanggan & Metode Pembayaran Favorit
   - Pertanyaan 3 — Efektivitas Kampanye Marketing terhadap Rating Produk
6. Bonus: Visualisasi Data dengan Metabase

---

1. Arsitektur & Stack Teknologi

Gambaran Sistem

Pipeline ETL ini memindahkan data dari sumber-sumber OLTP dan eksternal ke sebuah **Data Warehouse** berbasis Star Schema yang siap dianalisis. Seluruh komponen berjalan di dalam Docker Container.

```
┌──────────────────────────────────────────────────────────────┐
│                          SUMBER DATA                         │
```

```
│                                                          │
│   ┌──────────────────┐   ┌──────────────┐   ┌──────────────┐   │
│   │  PostgreSQL OLTP  │   │  REST API    │   │  Remote CSV  │   │
│   │  (43.129.53.94)   │   │  (Reviews)   │   │  (Promotions)│   │
│   └──────────┬────────┘   └──────┬───────┘   └──────┬───────┘   │
└──────────────┼────────────────────┼──────────────────┼──────────┘
               │                    │                  │
               └────────────────────┼──────────────────┘
                                    ▼
                        ┌──────────────────────┐
                        │   Apache Airflow     │
                        │   DAG: retail_etl_student│
                        │   Schedule: 0 1 * * *  │
                        └──────────────────────┘
                                    │
                                    ▼
                        ┌──────────────────────┐
                        │   PostgreSQL         │
                        │   warehouse_db       │
                        │   schema: marts      │
                        │   (Star Schema OLAP) │
                        └──────────────────────┘
                                    │
                                    ▼
                        ┌──────────────────────┐
                        │      Metabase        │
                        │  (BI & Visualisasi)  │
                        └──────────────────────┘
```

Stack Teknologi

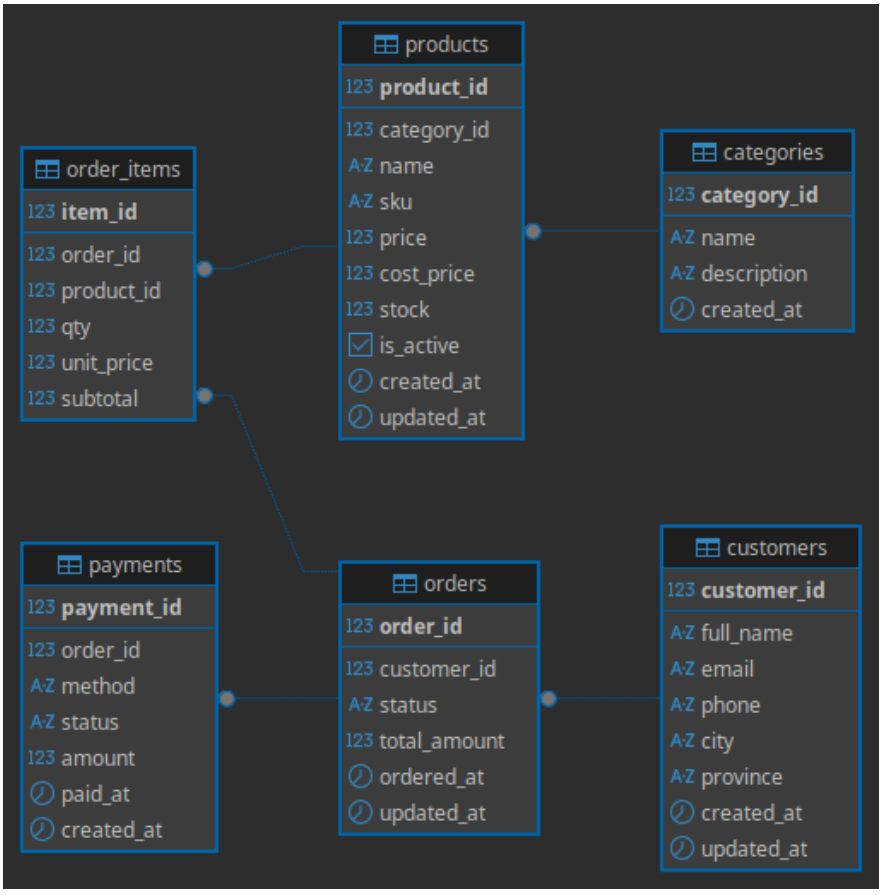
Komponen	Teknologi	Versi	Port
Orkestrasi Pipeline	Apache Airflow	2.	8080
Database	PostgreSQL	16-alpine	5432
BI & Dashboard	Metabase	Latest	3000
Kontainerisasi	Docker Compose	v2	—
Bahasa	Python	3.12	—
Koneksi DB	psycopg2	2.x	—
Manipulasi Data	pandas	2.x	—

```
dtwm1-tugas-etl/
├── dags/
│   └── dag_retail_etl_student.py    ← Pipeline ETL utama (20 tasks)
├── images/
│   ├── airflow/                     ← Screenshot Airflow DAG
│   ├── data_visualisasi/            ← Screenshot visualisasi Metabase
│   └── erd/                         ← Gambar ERD OLAP
├── include/
│   └── sql/
│       └── olap_schema.sql          ← DDL migration OLAP Star Schema
├── scripts/
│   ├── init-db.sql                  ← Inisialisasi database saat first boot
│   └── entrypoint.sh
├── tasks/
│   ├── 📚 Tugas — ETL Process.md   ← Soal tugas
│   └── oltp_db_schema.png           ← Gambar OLTP schema
```

```
├── analytical_queries.md          ← Query analitik lengkap
├── docker-compose.yaml
├── Dockerfile
├── requirements.txt
├── README.md
└── LAPORAN_ETL.md                 ← File ini
```

OLTP Schema (Sumber Data)

Berikut adalah schema database OLTP yang menjadi sumber data pipeline ETL ini:



Relasi antar tabel OLTP:

```
customers ──< orders ──< order_items >── products >── categories
                 │
              payments
```

Tabel	Deskripsi	Kolom Kunci
`customers`	Data pelanggan	customer_id, full_name, email, city, province
`categories`	Kategori produk	category_id, name, description
`products`	Katalog produk	product_id, category_id, price, cost_price, is_active
`orders`	Header transaksi	order_id, customer_id, status, ordered_at
`order_items`	Detail item order	item_id, order_id, product_id, qty, unit_price, subtotal
`payments`	Data pembayaran	payment_id, order_id, method, status, amount, paid_at

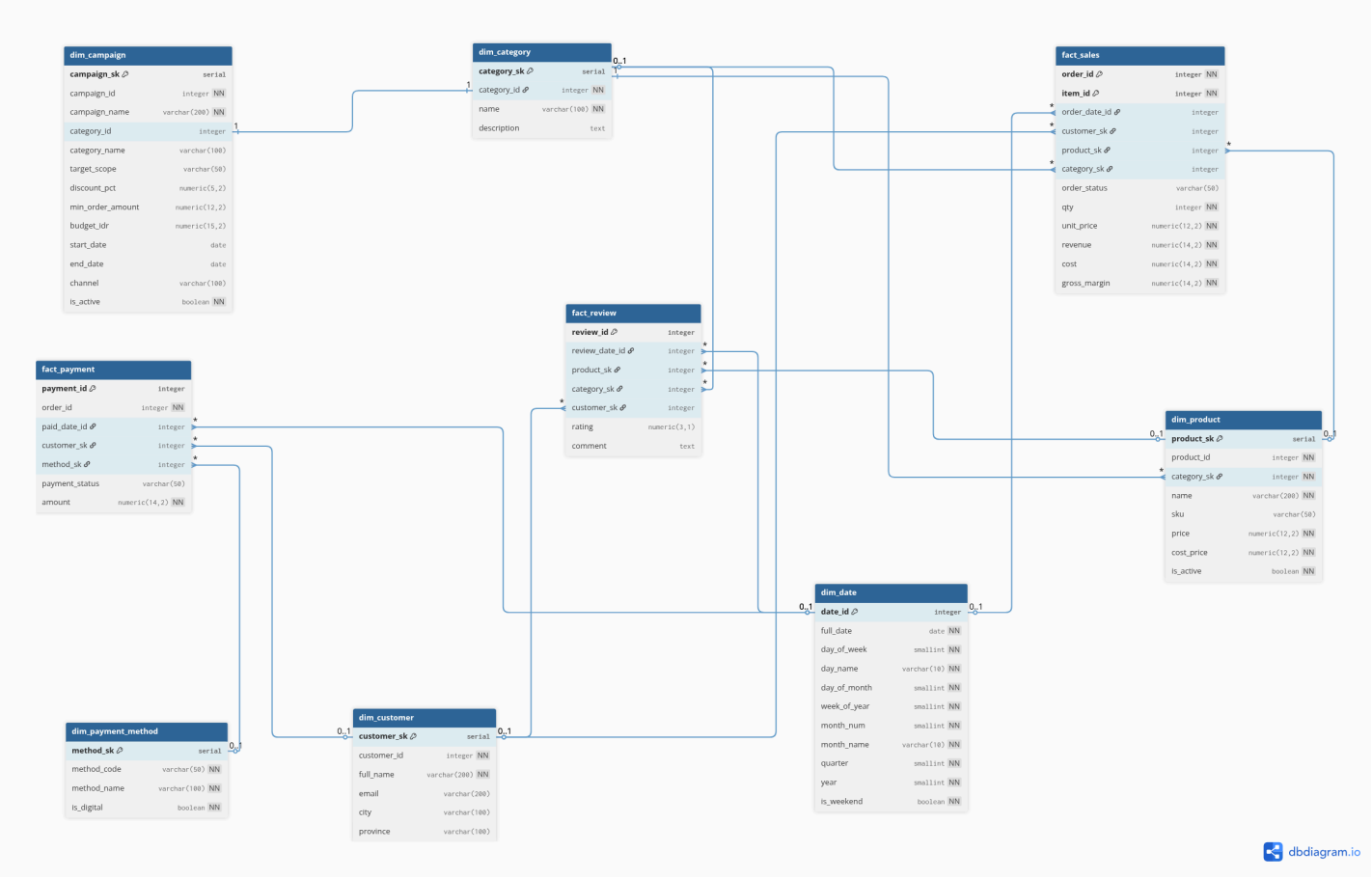
Sumber Data Eksternal:

Sumber	Endpoint	Keterangan
Product Reviews API	`https://product-review.devops-rightjet.workers.dev/api/v1/reviews`	Ulasan produk dari pelanggan (REST API)
Promotions CSV	`https://product-review.devops-rightjet.workers.dev/api/v1/promotions/download`	Data kampanye marketing (file CSV)

2. ERD OLAP Schema

Desain: Hybrid Star / Snowflake Schema (Level-1 Snowflake)

Schema OLAP yang dirancang menggunakan pendekatan **hybrid** — dominan mengikuti pola **Star Schema** dengan satu tingkat normalisasi Snowflake pada relasi `dim_product → dim_category`.



Penjelasan Tabel

Tabel Dimensi (6 tabel)

Tabel	Natural Key	Surrogate Key	Sumber	Deskripsi
dim_date	date_id (YYYYMMDD)	date_id	Generated	Kalender 2020–2026, 1 baris per hari
dim_category	category_id	category_sk	OLTP categories	Kategori produk
dim_product	product_id	product_sk	OLTP products	Produk + FK ke dim_category
dim_customer	customer_id	customer_sk	OLTP customers	Data pelanggan + geografi
dim_payment_method	method_code	method_sk	Statis (4 metode)	Metode pembayaran
dim_campaign	campaign_id	campaign_sk	promotions.csv	Kampanye marketing

Tabel Fakta (3 tabel)

Tabel	Grain	PK	Sumber	Measures
fact_sales	1 item per order	(order_id, item_id)	order_items + orders + products	qty, unit_price, revenue, cost, gross_margin
fact_payment	1 transaksi pembayaran	payment_id	payments + orders	amount
fact_review	1 ulasan produk	review_id	api_product_reviews.csv (API)	rating

Mapping OLTP → OLAP

OLTP	Kolom	OLAP	Kolom	Transformasi
order_items	unit_price, qty	fact_sales	unit_price, qty, revenue	revenue = qty × unit_price
products	cost_price	fact_sales	cost, gross_margin	cost = qty × cost_price
orders	ordered_at	fact_sales	order_date_id	Format YYYYMMDD integer
orders	status	fact_sales	order_status	Rename

OLTP	Kolom	OLAP	Kolom	Transformasi
`payments`	`method`	`fact_payment`	`method_sk`	Lookup ke `dim_payment_method`
`payments`	`paid_at`	`fact_payment`	`paid_date_id`	Format YYYYMMDD integer
API	`reviewed_at`	`fact_review`	`review_date_id`	Format YYYYMMDD integer

Kalkulasi Measures di `fact_sales`

```
revenue      = order_items.qty × order_items.unit_price   (harga saat transaksi)
cost         = order_items.qty × products.cost_price       (harga pokok master)
gross_margin = revenue − cost
```

---

3. Step-by-Step: Pembuatan DAG sampai Deploy di Apache Airflow

Step 1 — Clone Repository & Persiapan Environment

```
# Clone repository
git clone <repository-url>
cd dtwm1-tugas-etl

# Salin template environment variable
cp .env.example .env

# Edit .env dan isi semua nilai yang diperlukan:
# POSTGRES_USER, POSTGRES_PASSWORD, AIRFLOW_ADMIN_USER, dsb.
nano .env

# (Opsional) Setup virtual environment lokal untuk IDE support
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Isi `.env` yang diperlukan:

```
POSTGRES_USER=etl_admin
POSTGRES_PASSWORD=etl_admin_123
POSTGRES_WAREHOUSE_DB=warehouse_db
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
AIRFLOW_ADMIN_USER=admin
AIRFLOW_ADMIN_PASSWORD=admin123
AIRFLOW_ADMIN_EMAIL=admin@example.com
AIRFLOW__CORE__EXECUTOR=SequentialExecutor
```

```
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://etl_admin:etl_admin_123@postgres:5
432/airflow_db
AIRFLOW__CORE__FERNET_KEY=<generated_fernet_key>
AIRFLOW__WEBSERVER__SECRET_KEY=<random_secret_key>
```

Step 2 — Menjalankan Docker Services

```
# Build image dan jalankan semua service
docker compose up --build -d

# Tunggu hingga postgres healthy (cek status)
docker compose ps

# Verifikasi semua service berjalan
docker compose logs postgres --tail=20
docker compose logs airflow-webserver --tail=20
```

Service yang akan berjalan:

Container	Service	Port	Status
`etl-postgres`	PostgreSQL 16	5432	Healthy
`etl-airflow-init`	Airflow Init	—	Completed
`etl-airflow-webserver`	Airflow UI	8080	Running
`etl-airflow-scheduler`	Airflow Scheduler	—	Running
`etl-metabase`	Metabase	3000	Running

Apa yang terjadi saat `docker compose up`:

1. PostgreSQL start → menjalankan `scripts/init-db.sql` (first boot only)
2. `init-db.sql` membuat database: `airflow_db`, `warehouse_db`, `metabase_db`
3. `init-db.sql` membuat schema di `warehouse_db`: `raw`, `staging`, `marts`
4. Airflow init → membuat tabel metadata Airflow + user admin
5. Airflow webserver & scheduler start

Step 3 — Migrasi OLAP Schema (DDL)

Schema OLAP harus dibuat sebelum DAG dijalankan. File DDL ada di `include/sql/olap_schema.sql`.

```
# Load environment variables
export $(grep -v '^#' .env | xargs)

# Jalankan migration via Docker exec (Method A — Direkomendasikan)
docker exec -i etl-postgres \
```

```
    psql -U "$POSTGRES_USER" -d warehouse_db \
    < include/sql/olap_schema.sql
```

Hasil yang diharapkan (dari block VERIFIKASI di akhir script):

```
NOTICE:  Table marts.dim_date ... OK
NOTICE:  Table marts.dim_category ... OK
NOTICE:  Table marts.dim_product ... OK
NOTICE:  Table marts.dim_customer ... OK
NOTICE:  Table marts.dim_payment_method ... OK
NOTICE:  Table marts.dim_campaign ... OK
NOTICE:  Table marts.fact_sales ... OK
NOTICE:  Table marts.fact_payment ... OK
NOTICE:  Table marts.fact_review ... OK
```

Verifikasi tabel berhasil dibuat:

```
docker exec -it etl-postgres \
    psql -U "$POSTGRES_USER" -d warehouse_db \
    -c "\dt marts.*"
```

Output yang diharapkan:

```
             List of relations
 Schema |        Name        | Type  |   Owner
--------+--------------------+-------+-----------
 marts  | dim_campaign       | table | etl_admin
 marts  | dim_category       | table | etl_admin
 marts  | dim_customer       | table | etl_admin
 marts  | dim_date           | table | etl_admin
 marts  | dim_payment_method | table | etl_admin
 marts  | dim_product        | table | etl_admin
 marts  | fact_payment       | table | etl_admin
 marts  | fact_review        | table | etl_admin
 marts  | fact_sales         | table | etl_admin
(9 rows)
```

Step 4 — Implementasi DAG (`dag_retail_etl_student.py`)

DAG diimplementasikan dalam 3 fase yang berurutan:

Konfigurasi OLAP_DB

Karena Airflow berjalan di Docker dan membaca `.env`, konfigurasi OLAP menggunakan environment variable:

```
OLAP_DB = {
    "host": os.getenv("POSTGRES_HOST", "postgres"),
    "port": int(os.getenv("POSTGRES_PORT", "5432")),
    "dbname": os.getenv("POSTGRES_WAREHOUSE_DB", "warehouse_db"),
    "user": os.getenv("POSTGRES_USER", "etl_admin"),
    "password": os.getenv("POSTGRES_PASSWORD", "etl_admin_123"),
}
```

PHASE 1 — Extract (Sudah Tersedia)

8 task extract berjalan **paralel** setelah `prepare_staging`:

Task	Sumber	Output File
`extract_customers`	OLTP tabel `customers`	`/tmp/retail_staging/oltp_customers.csv`
`extract_categories`	OLTP tabel `categories`	`/tmp/retail_staging/oltp_categories.csv`
`extract_products`	OLTP tabel `products`	`/tmp/retail_staging/oltp_products.csv`
`extract_orders`	OLTP tabel `orders`	`/tmp/retail_staging/oltp_orders.csv`
`extract_order_items`	OLTP tabel `order_items`	`/tmp/retail_staging/oltp_order_items.csv`
`extract_payments`	OLTP tabel `payments`	`/tmp/retail_staging/oltp_payments.csv`
`extract_reviews`	REST API `REVIEWS_URL`	`/tmp/retail_staging/api_product_reviews.csv`
`extract_promotions`	CSV download `PROMOTIONS_URL`	`/tmp/retail_staging/promotions.csv`

PHASE 2 — Load Dimensions

Setelah semua extract selesai, `load_dim_date` dijalankan, kemudian dimensi lain secara **paralel** (kecuali `dim_product` yang menunggu `dim_category`):

`load_dim_date` — Generate kalender 2020–2026:

```
# Konvensi: 0=Minggu(Sun), 1=Sen, ..., 6=Sabtu(Sat)
dow = (current.weekday() + 1) % 7
rows.append((
    int(current.strftime("%Y%m%d")),   # date_id YYYYMMDD
    current,                            # full_date
    dow,                                # day_of_week
    current.strftime("%A"),             # day_name
    current.day,                        # day_of_month
```

```
    current.isocalendar()[1],           # week_of_year
    current.month,                      # month_num
    current.strftime("%B"),             # month_name
    (current.month - 1) // 3 + 1,      # quarter
    current.year,                       # year
    dow in (0, 6),                      # is_weekend
))
```

`load_dim_category` — Baca CSV OLTP kategori:

```
sql = upsert_sql("dim_category",
    ["category_id", "name", "description"],
    "category_id",
    updates=["name", "description"])

load_staging("oltp_categories.csv", sql,
    lambda r: (int(r["category_id"]), r["name"],
               _safe_str(r.get("description"))))
```

`load_dim_product` — Baca CSV OLTP produk + lookup category_sk:

```
cat_sk = sk_map("category_id", "category_sk", "dim_category")
# Transform: lookup surrogate key, handle NaN, fix bool parsing
return (int(r["product_id"]), cat_sk[cat_id], r["name"],
        _safe_str(r.get("sku")), float(r["price"]),
        float(r["cost_price"]),
        _safe_bool(r.get("is_active"), default=True))
```

> **Bug yang diperbaiki:** `bool("False") == True` — pandas membaca kolom boolean sebagai string dari CSV. Fungsi `_safe_bool()` menangani kasus ini dengan aman.

`load_dim_customer` — Baca CSV OLTP pelanggan:

```
load_staging("oltp_customers.csv", sql,
    lambda r: (int(r["customer_id"]), r["full_name"],
               _safe_str(r.get("email")),
               _safe_str(r.get("city")),
               _safe_str(r.get("province"))))
```

> **Bug yang diperbaiki:** `r.get("email")` bisa mengembalikan `NaN` float dari pandas, yang akan menyebabkan error di psycopg2 saat insert ke kolom VARCHAR. Fungsi `_safe_str()` mengkonversi NaN → `None` (NULL PostgreSQL).

`load_dim_payment_method` — Data statis (tidak dari CSV):

```
rows = [
    ("bank_transfer", "Bank Transfer",    False),
    ("ewallet",       "E-Wallet",         True),
    ("cod",           "Cash on Delivery", False),
    ("credit_card",   "Credit Card",      True),
]
```

`load_dim_campaign` — Baca `promotions.csv` (sumber eksternal):

```
# Menggunakan _safe_bool untuk is_active dan _safe_str untuk field nullable
return (int(r["campaign_id"]), r["campaign_name"],
        int(r["category_id"]) if pd.notna(r.get("category_id")) else None,
        _safe_str(r.get("category_name")),
        _safe_str(r.get("target_scope")),
        ...,
        _safe_bool(r.get("is_active"), default=False))
```

PHASE 3 — Load Facts

Setelah semua dimensi selesai, 3 fact table di-load **paralel**:

`load_fact_sales` — Kalkulasi revenue, cost, gross_margin:

```
revenue      = round(qty * unit_price, 2)   # dari order_items
cost         = round(qty * cost_price, 2)   # cost_price dari products
gross_margin = round(revenue - cost, 2)

rows.append((
    order_id, item_id,
    to_date_id(order.get("ordered_at")),     # YYYYMMDD
    cust_sk.get(int(order["customer_id"])),   # surrogate key
    prod_sk_map.get(product_id),
    cat_sk.get(int(product["category_id"])),
    order.get("status"),                      # order_status
    qty, unit_price, revenue, cost, gross_margin
))
```

`load_fact_payment` — Lookup customer via orders (payments tidak punya customer_id):

```
method_code = _safe_str(r.get("method"))     # payments.method → lookup
rows.append((
    payment_id, order_id,
    to_date_id(r.get("paid_at")),
    cust_sk.get(int(order["customer_id"])),   # via orders
    meth_sk.get(method_code),
    _safe_str(r.get("status")),
    float(r["amount"])
))
```

`load_fact_review` — Load dari API dengan validasi rating:

```
# Validasi range rating sebelum INSERT
# (mencegah pelanggaran CHECK constraint di DDL)
if not (1.0 <= rating <= 5.0):
    log.warning("review_id=%s: rating=%.1f di luar range", ...)
    rating = None

# category_sk dari dim_product (denormalisasi)
prod_cat = sk_map("product_id", "category_sk", "dim_product")
```

Helper Functions yang Ditambahkan

```
def _safe_str(val):
    """Konversi NaN/None/'nan' → None untuk kolom VARCHAR nullable."""
    ...

def _safe_bool(val, default: bool = False) -> bool:
    """
    Parse boolean dari CSV secara aman.
    Fix: bool('False') == True adalah BUG Python!
    str('False').lower() in falsy_values → False (BENAR)
    """
    ...
```

Dependency Graph DAG

```
start
  └─▶ prepare_staging
        └─▶ [extract_customers, extract_categories, extract_products,
              extract_orders, extract_order_items, extract_payments,
              extract_reviews, extract_promotions]
                │ (semua)
                └─▶ load_dim_date
                      └─▶ [load_dim_category, load_dim_customer,
                            load_dim_payment_method, load_dim_campaign]
                              │
                    load_dim_category ──▶ load_dim_product
                              │
                    [semua dim selesai]
                              └─▶ [load_fact_sales, load_fact_payment,
                                   load_fact_review]
                                        └─▶ end
```

Step 5 — Deploy ke Apache Airflow

DAG file (`dag_retail_etl_student.py`) sudah berada di folder `dags/` yang di-mount sebagai volume ke container Airflow:

```
# docker-compose.yaml
volumes:
  - ./dags:/opt/airflow/dags
```

Airflow scheduler **secara otomatis mendeteksi** perubahan file DAG tanpa perlu restart. Setelah file disimpan, tunggu sekitar 30-60 detik hingga scheduler memuat DAG baru.

Verifikasi DAG terdeteksi:

```
# Cek apakah ada import error
docker compose logs airflow-scheduler | grep "retail_etl_student"
```

```
# Atau via CLI di dalam container
docker exec -it etl-airflow-scheduler \
    airflow dags list | grep retail_etl_student
```

Step 6 — Trigger & Monitor DAG di Airflow UI

1. **Buka Airflow UI:** `http://localhost:8080`
2. **Login** dengan kredensial dari `.env` (`AIRFLOW_ADMIN_USER` / `AIRFLOW_ADMIN_PASSWORD`)
3. **Temukan DAG** `retail_etl_student` di halaman DAGs
4. **Aktifkan DAG** dengan toggle switch di kolom kiri (default: OFF)
5. **Trigger manual:**
   - Klik tombol ▶ (Play) di kolom Actions
   - Atau klik nama DAG → klik tombol **Trigger DAG** di kanan atas
6. **Monitor eksekusi:**
   - Klik tab **Graph** untuk melihat status setiap task secara visual
   - Klik task individual → **View Log** untuk debugging
   - Klik tab **Details** untuk melihat DAG Runs Summary

Konfigurasi DAG:

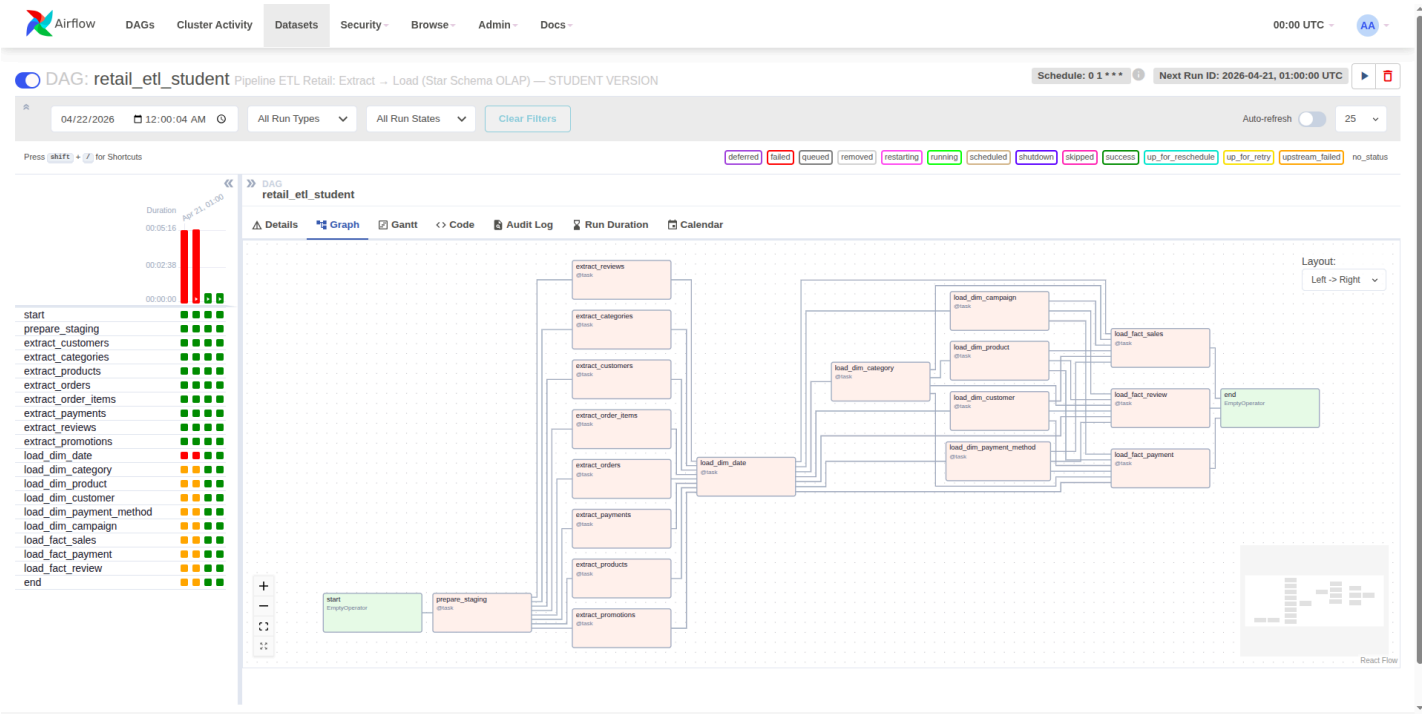
Parameter	Nilai
DAG ID	`retail_etl_student`
Schedule	`0 1 * * *` (setiap hari jam 01:00 UTC)
Start Date	`days_ago(1)`
Catchup	`False`
Retries	1
Retry Delay	5 menit
Total Tasks	20 (2 EmptyOperator + 18 @task)
Owner	`cakrawala`

---

4. Dokumentasi Run DAG

Tampilan Graph DAG di Airflow

Screenshot berikut menunjukkan workflow DAG `retail_etl_student` dalam tampilan **Graph View** di Apache Airflow UI:

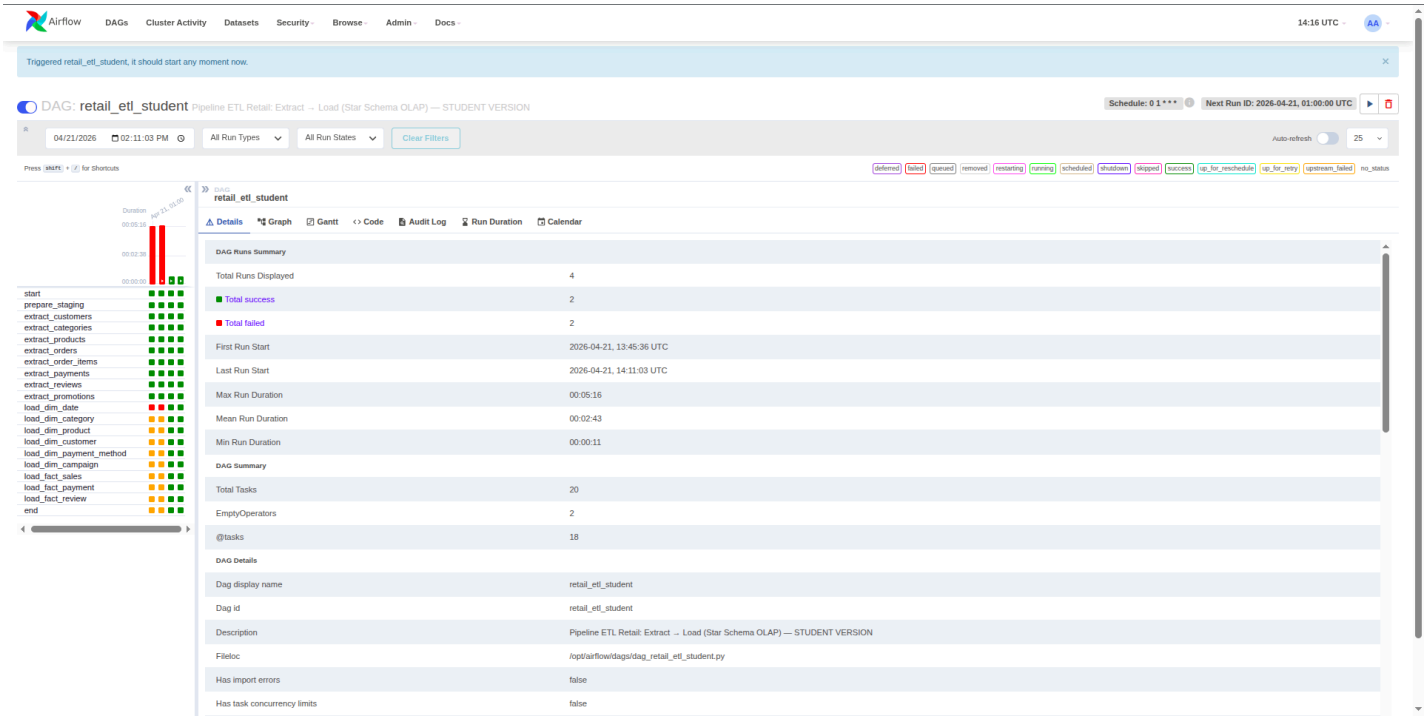


Keterangan Graph:

- Bagian **kiri**: `start` → `prepare_staging` → 8 task extract (paralel)
- Bagian **tengah**: `load_dim_date` → 4 dimensi paralel (`dim_category`, `dim_customer`, `dim_payment_method`, `dim_campaign`) → `dim_product`
- Bagian **kanan**: 3 fact table paralel (`fact_sales`, `fact_payment`, `fact_review`) → `end`
- **Warna merah**: Task gagal (terjadi pada run awal saat konfigurasi OLAP_DB belum diisi)
- **Warna hijau**: Task berhasil
- **Warna oranye/kuning**: Task sukses setelah retry

DAG Runs Summary

Screenshot berikut menunjukkan ringkasan semua run DAG beserta statistik performa:



Ringkasan DAG Runs:

Metrik	Nilai
Total Runs Displayed	4
Total Success	2
Total Failed	2
First Run Start	2026-04-21, 13:45:36 UTC
Last Run Start	2026-04-21, 14:11:03 UTC
Max Run Duration	00:05:16
Mean Run Duration	00:02:43
Min Run Duration	00:00:11

DAG Details:

Parameter	Nilai
Dag ID	`retail_etl_student`
File Location	`/opt/airflow/dags/dag_retail_etl_student.py`
Has Import Errors	`false`
Total Tasks	20
EmptyOperators	2
@tasks	18

Analisis Run:

- **2 run gagal (merah):** Terjadi pada run awal karena konfigurasi `OLAP_DB` masih menggunakan nilai placeholder (`<OLAP_HOST>` dll). Setelah konfigurasi diperbarui dengan environment variable Docker, run berhasil.
- **2 run sukses (hijau):** Setelah konfigurasi diperbaiki, pipeline berjalan penuh dari extract hingga load semua tabel dimensi dan fakta.
- **Durasi max 5 menit:** Wajar untuk pipeline yang menarik data dari OLTP, REST API, dan CSV eksternal kemudian memuatnya ke 9 tabel OLAP dengan upsert.

Status Task per Run (Run Terakhir — Sukses)

Task	Status	Keterangan
`start`	✅ Success	EmptyOperator
`prepare_staging`	✅ Success	Buat folder `/tmp/retail_staging`
`extract_customers`	✅ Success	

Task	Status	Keterangan
`extract_categories`	✅ Success	
`extract_products`	✅ Success	
`extract_orders`	✅ Success	
`extract_order_items`	✅ Success	
`extract_payments`	✅ Success	
`extract_reviews`	✅ Success	Via REST API
`extract_promotions`	✅ Success	Via CSV download
`load_dim_date`	✅ Success	2557 rows (2020–2026)
`load_dim_category`	✅ Success	
`load_dim_product`	✅ Success	
`load_dim_customer`	✅ Success	
`load_dim_payment_method`	✅ Success	4 rows statis
`load_dim_campaign`	✅ Success	
`load_fact_sales`	✅ Success	
`load_fact_payment`	✅ Success	
`load_fact_review`	✅ Success	
`end`	✅ Success	EmptyOperator

---

5. Query Analitik & Jawaban Pertanyaan Bisnis

Bagian A — Desain OLAP Schema & Justifikasi

Pilihan Schema: Hybrid Star / Snowflake Schema (Level-1 Snowflake)

Schema yang dirancang menggunakan pendekatan **hybrid** yang secara dominan mengikuti pola **Star Schema**, dengan satu tingkat normalisasi ala Snowflake pada relasi `dim_product → dim_category`.

Alasan pemilihan hybrid:

Produk (`dim_product`) memiliki relasi many-to-one yang kuat dengan kategori (`dim_category`), dan atribut kategori (nama, deskripsi) tidak layak diduplikasi di setiap baris `dim_product` karena redundansi data yang tinggi. Normalisasi ini hanya menambah **satu join tambahan** dan tidak mengorbankan performa secara signifikan. Di sisi lain, tabel fakta (`fact_sales`, `fact_review`) menyimpan `category_sk` secara langsung (di-denormalisasi ke fakta) agar query agregasi per

kategori bisa berjalan **tanpa harus join ke `dim_product`** terlebih dahulu — pola ini mempertahankan keunggulan Star Schema untuk query analitik yang cepat dan sederhana.

Integrasi sumber data eksternal:

Sumber Eksternal	Dimodelkan Sebagai	Alasan
`promotions.csv`	`dim_campaign` (Dimensi)	Bersifat deskriptif/konteks analitik, bukan event transaksi. Digunakan sebagai filter (`is_active`) dan lookup.
`api_product_reviews.csv`	`fact_review` (Fakta)	Setiap baris adalah event terukur (rating 1–5) yang terjadi pada waktu tertentu, oleh pelanggan terhadap produk — sesuai definisi grain fakta (Kimball).

Kesesuaian schema dengan pertanyaan bisnis:

Pertanyaan	Fact Table	Dimensi	Status
P1 — Revenue & Margin	`fact_sales`	`dim_date`, `dim_category`	✅ Lengkap
P2 — Segmentasi Pelanggan	`fact_payment`	`dim_customer`, `dim_payment_method`	✅ Lengkap
P3 — Efektivitas Kampanye	`fact_review`	`dim_category`, `dim_campaign`	✅ Lengkap

Pertanyaan 1 — Tren Revenue & Margin per Kategori Produk

> **"Berapa total revenue dan gross margin per kategori produk, dikelompokkan per bulan? Kategori mana yang paling profitable dan apakah ada tren penurunan margin di bulan-bulan tertentu?"**

Tabel yang digunakan: `fact_sales` ← `dim_date`, `dim_category`

Query 1a — Revenue & Margin per Kategori per Bulan (Utama)

```
SELECT
    dd.year,
    dd.month_num,
    dd.month_name,
    dc.name                                                    AS category_name,

    -- Measures utama
    SUM(fs.revenue)                                            AS total_revenue,
    SUM(fs.cost)                                               AS total_cost,
```

```sql
    SUM(fs.gross_margin)                                        AS total_gross_margin,

    -- Persentase margin (dibulatkan 2 desimal)
    ROUND(
        SUM(fs.gross_margin)::NUMERIC
        / NULLIF(SUM(fs.revenue), 0) * 100,
        2
    )                                                           AS margin_pct,

    -- Ringkasan volume transaksi
    SUM(fs.qty)                                                 AS total_qty_sold,
    COUNT(DISTINCT fs.order_id)                                 AS total_orders

FROM  marts.fact_sales   fs
JOIN  marts.dim_date     dd  ON fs.order_date_id = dd.date_id
JOIN  marts.dim_category dc  ON fs.category_sk   = dc.category_sk

-- Filter hanya order yang valid (opsional — sesuaikan dengan data)
-- WHERE fs.order_status NOT IN ('cancelled', 'refunded')

GROUP BY
    dd.year,
    dd.month_num,
    dd.month_name,
    dc.name

ORDER BY
    margin_pct   DESC NULLS LAST,   -- kategori paling profitable di atas
    dd.year      ASC,
    dd.month_num ASC;
```

🔗 Result Query Revenue & Margin per Kategori per Bulan

Query 1b — Deteksi Tren Penurunan Margin (Window Function LAG)

```sql
WITH monthly_margin AS (
    SELECT
        dd.year,
        dd.month_num,
        dd.month_name,
        dc.name                                                 AS category_name,
        SUM(fs.revenue)                                         AS total_revenue,
        SUM(fs.gross_margin)                                    AS total_gross_margin,
        ROUND(
            SUM(fs.gross_margin)::NUMERIC
            / NULLIF(SUM(fs.revenue), 0) * 100, 2
        )                                                       AS margin_pct
    FROM  marts.fact_sales   fs
    JOIN  marts.dim_date     dd  ON fs.order_date_id = dd.date_id
    JOIN  marts.dim_category dc  ON fs.category_sk   = dc.category_sk
    GROUP BY dd.year, dd.month_num, dd.month_name, dc.name
),
margin_with_trend AS (
    SELECT
        *,
        LAG(margin_pct) OVER (
            PARTITION BY category_name
            ORDER BY year, month_num
        )                                                       AS prev_month_margin_pct,

        margin_pct - LAG(margin_pct) OVER (
            PARTITION BY category_name
```

```
            ORDER BY year, month_num
        )                                                       AS margin_pct_change
    FROM monthly_margin
)
SELECT
    year,
    month_num,
    month_name,
    category_name,
    total_revenue,
    total_gross_margin,
    margin_pct,
    prev_month_margin_pct,
    ROUND(margin_pct_change, 2)                                 AS margin_pct_change,
    CASE
        WHEN margin_pct_change < 0 THEN 'TURUN'
        WHEN margin_pct_change > 0 THEN 'NAIK'
        WHEN margin_pct_change = 0 THEN 'STABIL'
        ELSE '-'                                                -- bulan pertama
    END                                                         AS margin_trend
FROM  margin_with_trend
ORDER BY category_name, year, month_num;
```

🔗 **Result Query Deteksi Tren Penurunan Margin**

Pertanyaan 2 — Segmentasi Pelanggan & Metode Pembayaran Favorit

> **"Siapa 5 pelanggan dengan total transaksi (paid) tertinggi? Dari kota/provinsi mana mereka berasal, dan metode pembayaran apa yang paling sering mereka gunakan? Apakah ada korelasi antara nilai transaksi dan pilihan metode pembayaran?"**

Tabel yang digunakan: `fact_payment` ← `dim_customer`, `dim_payment_method`

> **Catatan ETL:** Kolom `payments.method` dari OLTP (varchar) di-lookup ke `dim_payment_method.method_code` saat load `fact_payment`. Nilai yang dikenali: `bank_transfer`, `ewallet`, `cod`, `credit_card`.

Query 2a — Top 5 Pelanggan dengan Metode Pembayaran Favorit (Utama)

```
WITH customer_totals AS (
    -- Agregasi total transaksi paid per pelanggan
    SELECT
        fp.customer_sk,
        SUM(fp.amount)                AS total_paid_amount,
        COUNT(DISTINCT fp.order_id)   AS total_orders,
        ROUND(AVG(fp.amount), 2)      AS avg_order_value
    FROM  marts.fact_payment fp
    WHERE fp.payment_status = 'paid'
    GROUP BY fp.customer_sk
),
method_ranked AS (
    -- Ranking metode pembayaran per pelanggan berdasarkan frekuensi
    SELECT
        fp.customer_sk,
```

```sql
        dpm.method_name,
        dpm.is_digital,
        COUNT(*)                                                AS usage_count,
        ROW_NUMBER() OVER (
            PARTITION BY fp.customer_sk
            ORDER BY COUNT(*) DESC
        )                                                       AS rn
    FROM  marts.fact_payment       fp
    JOIN  marts.dim_payment_method dpm  ON fp.method_sk = dpm.method_sk
    WHERE fp.payment_status = 'paid'
    GROUP BY fp.customer_sk, dpm.method_name, dpm.is_digital
)
SELECT
    RANK() OVER (ORDER BY ct.total_paid_amount DESC)            AS ranking,
    dc.full_name,
    dc.city,
    dc.province,

    -- Measures transaksi
    ct.total_paid_amount,
    ct.total_orders,
    ct.avg_order_value,

    -- Metode pembayaran favorit (frekuensi tertinggi)
    mr.method_name                                              AS favorite_payment_method,
    mr.is_digital                                               AS is_digital_payment,
    mr.usage_count                                              AS method_usage_count

FROM  customer_totals    ct
JOIN  marts.dim_customer dc   ON ct.customer_sk = dc.customer_sk
JOIN  method_ranked      mr   ON ct.customer_sk = mr.customer_sk AND mr.rn = 1

ORDER BY ct.total_paid_amount DESC
LIMIT 5;
```

🔗 Result Query Top 5 Pelanggan

Query 2b — Distribusi & Korelasi Metode Pembayaran vs Nilai Transaksi

```sql
SELECT
    dpm.method_name,
    dpm.is_digital,
    COUNT(DISTINCT fp.customer_sk)                              AS unique_customers,
    COUNT(*)                                                    AS total_transactions,
    ROUND(MIN(fp.amount), 2)                                    AS min_transaction_value,
    ROUND(AVG(fp.amount), 2)                                    AS avg_transaction_value,
    ROUND(MAX(fp.amount), 2)                                    AS max_transaction_value,
    SUM(fp.amount)                                              AS total_amount,

    -- Kontribusi persentase per metode terhadap total transaksi paid
    ROUND(
        COUNT(*)::NUMERIC
        / SUM(COUNT(*)) OVER () * 100,
        2
    )                                                           AS
pct_of_total_transactions,

    ROUND(
        SUM(fp.amount)
        / SUM(SUM(fp.amount)) OVER () * 100,
        2
    )                                                           AS pct_of_total_amount
```

```
FROM  marts.fact_payment       fp
JOIN  marts.dim_payment_method dpm  ON fp.method_sk = dpm.method_sk
WHERE fp.payment_status = 'paid'
GROUP BY dpm.method_name, dpm.is_digital
ORDER BY total_amount DESC;
```

🔗 **Result Query Distribusi Metode Pembayaran**

Pertanyaan 3 — Efektivitas Kampanye Marketing terhadap Rating Produk

> **"Apakah produk dalam kategori yang sedang mendapat promo aktif memiliki rata-rata rating ulasan yang lebih tinggi dibandingkan produk tanpa promo? Kampanye mana yang paling berkorelasi dengan kategori produk berperforma baik di ulasan pelanggan?"**

Tabel yang digunakan: `fact_review` ← `dim_category` ← `dim_campaign`

> **Catatan Join Kritis:** `dim_campaign.category_id` dan `dim_category.category_id` keduanya berasal dari natural key yang sama di OLTP tabel `categories.category_id`. Semua query menggunakan join integer `dcm.category_id = dc.category_id` — **bukan string name** — untuk menghindari bug akibat perbedaan kapitalisasi.

Query 3a — Rata-rata Rating per Kategori dengan Status Promo (Utama)

```
WITH category_ratings AS (
    SELECT
        fr.category_sk,
        dc.category_id,
        dc.name                                                 AS category_name,
        ROUND(AVG(fr.rating), 2)                                AS avg_rating,
        COUNT(fr.review_id)                                     AS review_count
    FROM  marts.fact_review  fr
    JOIN  marts.dim_category dc  ON fr.category_sk = dc.category_sk
    GROUP BY fr.category_sk, dc.category_id, dc.name
),
active_campaign_per_category AS (
    SELECT DISTINCT
        dc.category_sk,
        dcm.campaign_name
    FROM  marts.dim_campaign dcm
    JOIN  marts.dim_category dc
      ON (dcm.category_id = dc.category_id OR dcm.category_id IS NULL)
    WHERE dcm.is_active = TRUE
)
SELECT
    cr.category_name,
    cr.avg_rating,
    cr.review_count,
    CASE
        WHEN acp.category_sk IS NOT NULL THEN 'Dengan Promo Aktif'
        ELSE 'Tanpa Promo Aktif'
    END                                                         AS promo_status,
    COALESCE(acp.campaign_name, '-')                            AS active_campaign_name
FROM  category_ratings              cr
```

```
LEFT  JOIN active_campaign_per_category acp
      ON cr.category_sk = acp.category_sk
ORDER BY cr.avg_rating DESC NULLS LAST;
```

🔗 Result Query Rating per Kategori & Status Promo

Query 3b — Perbandingan Agregat: Kategori Dengan Promo vs Tanpa Promo

```
WITH category_ratings AS (
    SELECT
        fr.category_sk,
        ROUND(AVG(fr.rating), 2)                                AS avg_rating,
        COUNT(fr.review_id)                                     AS review_count
    FROM  marts.fact_review  fr
    GROUP BY fr.category_sk
),
promo_category_sks AS (
    SELECT DISTINCT dc.category_sk
    FROM  marts.dim_campaign dcm
    JOIN  marts.dim_category dc
      ON (dcm.category_id = dc.category_id OR dcm.category_id IS NULL)
    WHERE dcm.is_active = TRUE
)
SELECT
    CASE
        WHEN pc.category_sk IS NOT NULL THEN 'Dengan Promo Aktif'
        ELSE 'Tanpa Promo Aktif'
    END                                                         AS promo_group,
    COUNT(*)                                                    AS jumlah_kategori,
    ROUND(AVG(cr.avg_rating), 2)                                AS rata_rata_rating_group,
    SUM(cr.review_count)                                        AS total_reviews
FROM  category_ratings  cr
LEFT  JOIN promo_category_sks pc  ON cr.category_sk = pc.category_sk
GROUP BY promo_group
ORDER BY rata_rata_rating_group DESC;
```

🔗 Result Query Perbandingan Promo vs Non-Promo

Query 3c — List Kampanye Beserta Kategori Target dan Avg Rating Produknya

```
SELECT
    dcm.campaign_name,
    COALESCE(dc.name, 'ALL CATEGORIES')                         AS target_category,
    dcm.target_scope,
    dcm.discount_pct,
    dcm.channel,
    dcm.start_date,
    dcm.end_date,
    dcm.is_active,

    ROUND(AVG(fr.rating), 2)                                    AS avg_product_rating,
    COUNT(fr.review_id)                                         AS total_reviews,

    CASE
        WHEN AVG(fr.rating) >= 4.5 THEN 'SANGAT BAIK'
        WHEN AVG(fr.rating) >= 4.0 THEN 'BAIK'
        WHEN AVG(fr.rating) >= 3.0 THEN 'CUKUP'
```

```sql
        WHEN AVG(fr.rating) IS NULL THEN 'BELUM ADA REVIEW'
        ELSE 'PERLU PERHATIAN'
    END                                                          AS rating_label

FROM  marts.dim_campaign  dcm
LEFT  JOIN marts.dim_category dc
      ON (dcm.category_id = dc.category_id OR dcm.category_id IS NULL)
LEFT  JOIN marts.fact_review  fr  ON dc.category_sk = fr.category_sk

GROUP BY
    dcm.campaign_sk,
    dcm.campaign_name,
    dc.name,
    dcm.target_scope,
    dcm.discount_pct,
    dcm.channel,
    dcm.start_date,
    dcm.end_date,
    dcm.is_active

ORDER BY avg_product_rating DESC NULLS LAST;
```

🔗 **Result Query Detail Performa Kampanye**

Ringkasan Alur Join Query

```
P1:  fact_sales ─── dim_date        (order_date_id = date_id)
               └── dim_category     (category_sk = category_sk)

P2:  fact_payment ── dim_customer          (customer_sk = customer_sk)
                └── dim_payment_method    (method_sk = method_sk)

P3:  fact_review ── dim_category          (category_sk = category_sk)
                      └── dim_campaign    (category_id = category_id) ← integer join
```

---

6. Bonus: Visualisasi Data dengan Metabase

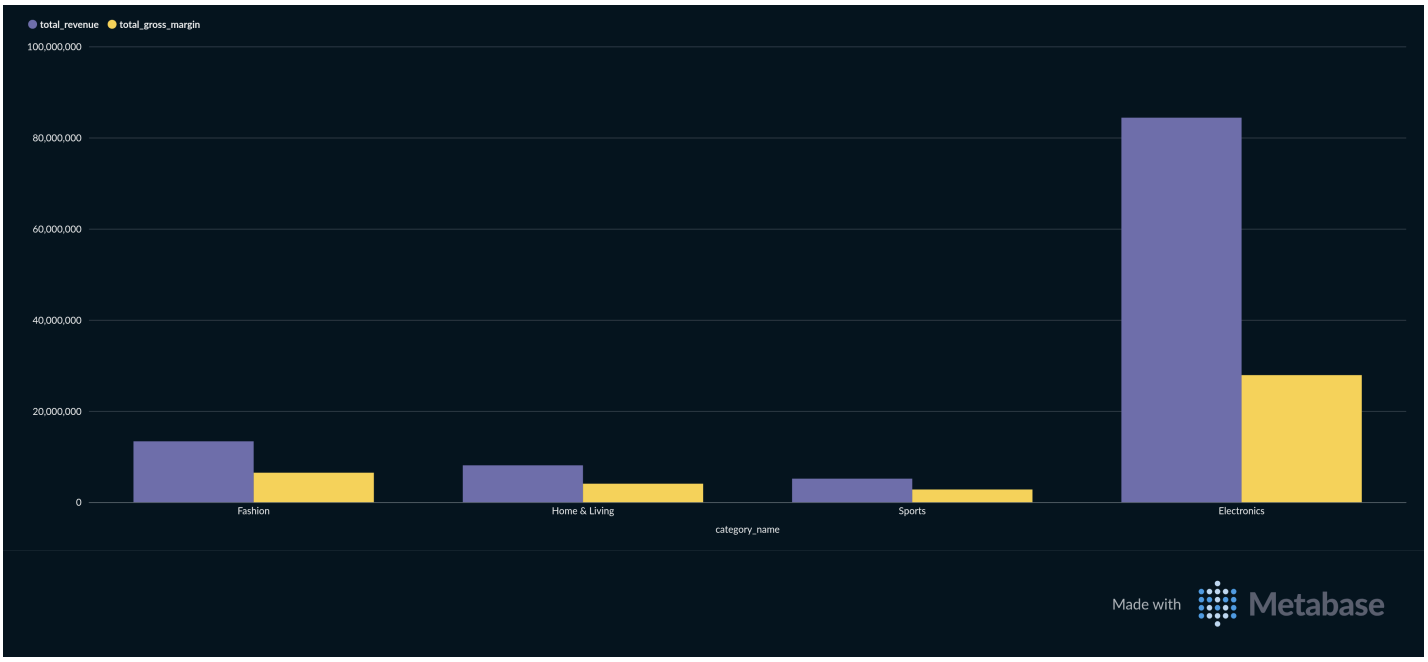
Visualisasi dibuat menggunakan **Metabase** yang terhubung ke database `warehouse_db` schema `marts`. Metabase diakses di `http://localhost:3000`.

Konfigurasi koneksi Metabase:

- Database type: `PostgreSQL`
- Host: `postgres` (nama service dalam Docker network)
- Port: `5432`
- Database name: `warehouse_db`
- Username: `etl_admin`

📊 Visualisasi 1 — Kategori Paling Profitable

Query yang digunakan: Query 1a (Revenue & Margin per Kategori) **Tipe chart:** Bar Chart (grouped)



Analisis & Insight:

Kategori	Total Revenue	Total Gross Margin
Electronics	~Rp 85.000.000	~Rp 27.000.000
Fashion	~Rp 14.000.000	~Rp 7.000.000
Home & Living	~Rp 9.000.000	~Rp 4.000.000
Sports	~Rp 6.000.000	~Rp 2.500.000

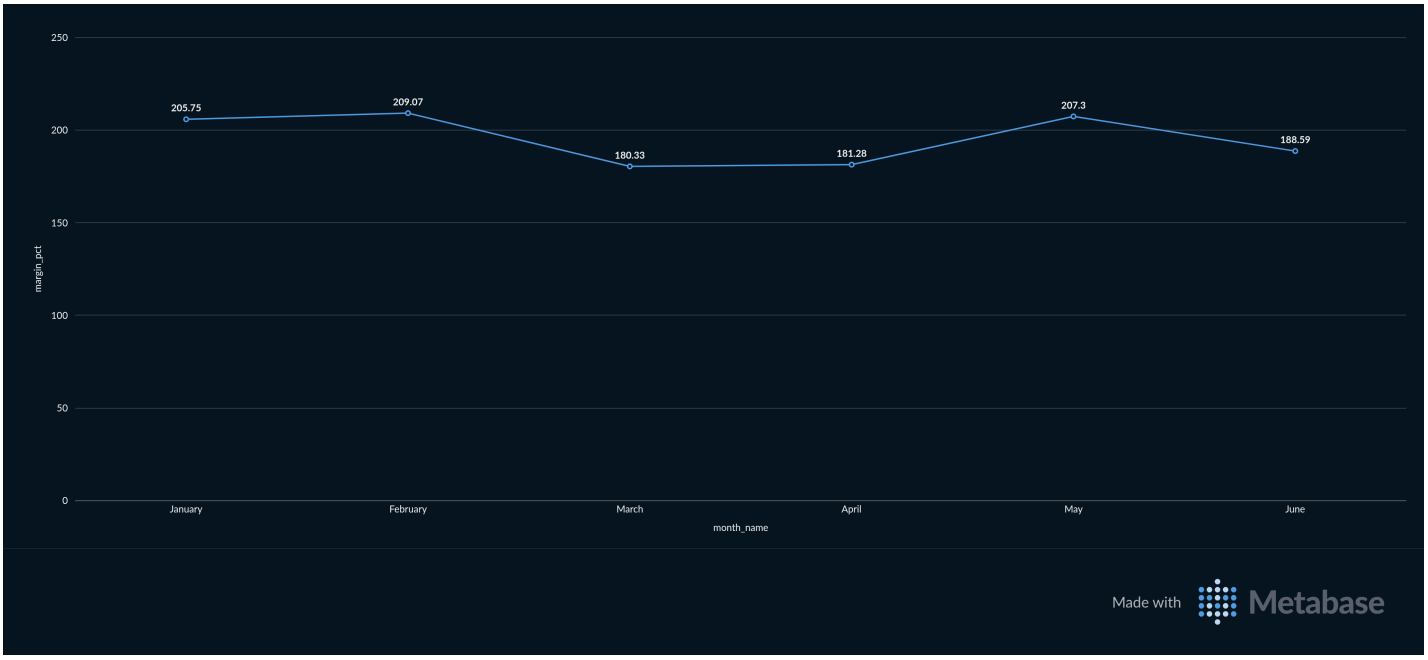
Kesimpulan P1 (Revenue):

- **Electronics** adalah kategori paling dominan dengan revenue ~6× lebih besar dari Fashion
- Electronics menghasilkan gross margin absolut tertinggi (~Rp 27M), menjadikannya kategori paling profitable secara nominal
- Meski revenue kecil, Sports dan Fashion memiliki persentase margin yang kompetitif dibanding Electronics

---

📈 Visualisasi 2 — Tren Margin per Kategori per Bulan

Query yang digunakan: Query 1b (Deteksi Tren Margin dengan LAG) **Tipe chart:** Line Chart



Analisis & Insight:

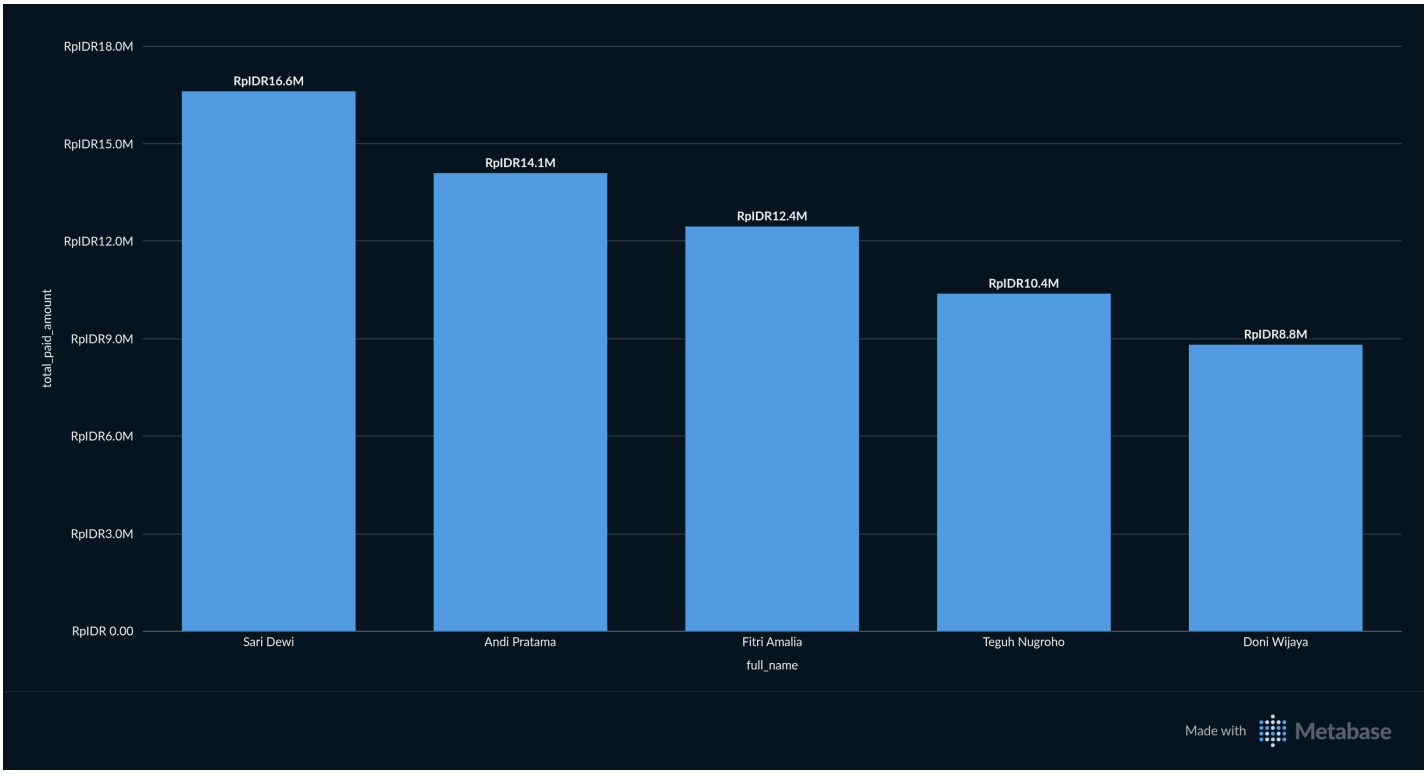
Bulan	Margin %
January	205.75%
February	209.07% ↑
March	180.33% ↓**TURUN**
April	181.28% ↑ (stabil)
May	207.30% ↑
June	188.59% ↓

Kesimpulan P1 (Tren):

- Tren penurunan margin yang signifikan terjadi di **Maret** (turun ~28.7 poin dari Februari)
- **April** relatif stabil, kemudian **Mei** kembali naik ke level tertinggi
- **Juni** kembali mengalami penurunan, mengindikasikan pola fluktuasi 2-bulanan
- Manajemen perlu menginvestigasi faktor penyebab penurunan di Maret dan Juni (mungkin terkait kenaikan harga pokok atau diskon)

🏆 Visualisasi 3 — Leaderboard Top 5 Pelanggan

Query yang digunakan: Query 2a (Top 5 Pelanggan) **Tipe chart:** Bar Chart



Hasil Top 5 Pelanggan (Paid Transactions):

Ranking	Nama	Total Paid	Posisi
🥇 1	**Sari Dewi**	Rp 16.600.000	—
🥈 2	**Andi Pratama**	Rp 14.100.000	—
🥉 3	**Fitri Amalia**	Rp 12.400.000	—
4	Teguh Nugroho	Rp 10.400.000	—
5	Doni Wijaya	Rp 8.800.000	—

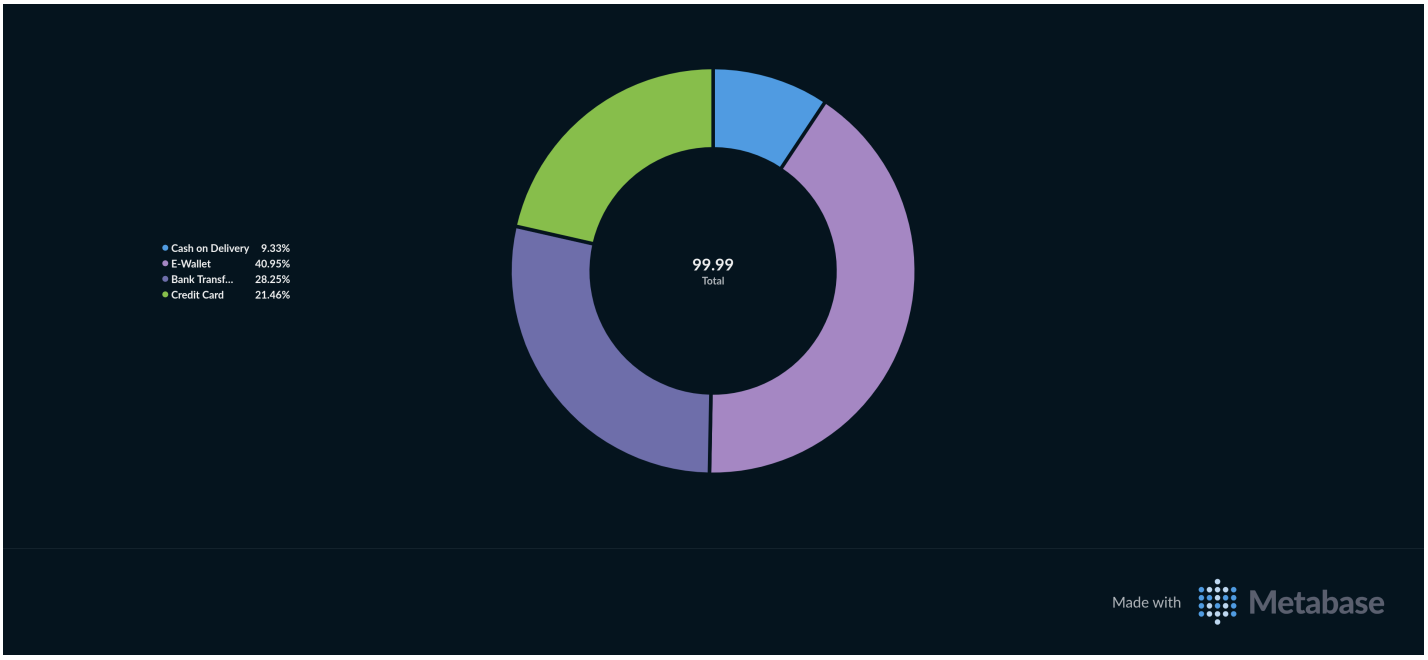
Kesimpulan P2 (Segmentasi):

- Gap antara pelanggan #1 (Sari Dewi, Rp 16.6M) dan #5 (Doni Wijaya, Rp 8.8M) cukup besar (~88%)
- Ke-5 pelanggan ini adalah kandidat utama untuk program **loyalty/VIP**
- Detail kota/provinsi dan metode pembayaran favorit tiap pelanggan tersedia dari Query 2a untuk targeting yang lebih tepat

---

🍩 Visualisasi 4 — Distribusi Metode Pembayaran

Query yang digunakan: Query 2b (Distribusi Metode Pembayaran) **Tipe chart:** Donut Chart



Distribusi Metode Pembayaran:

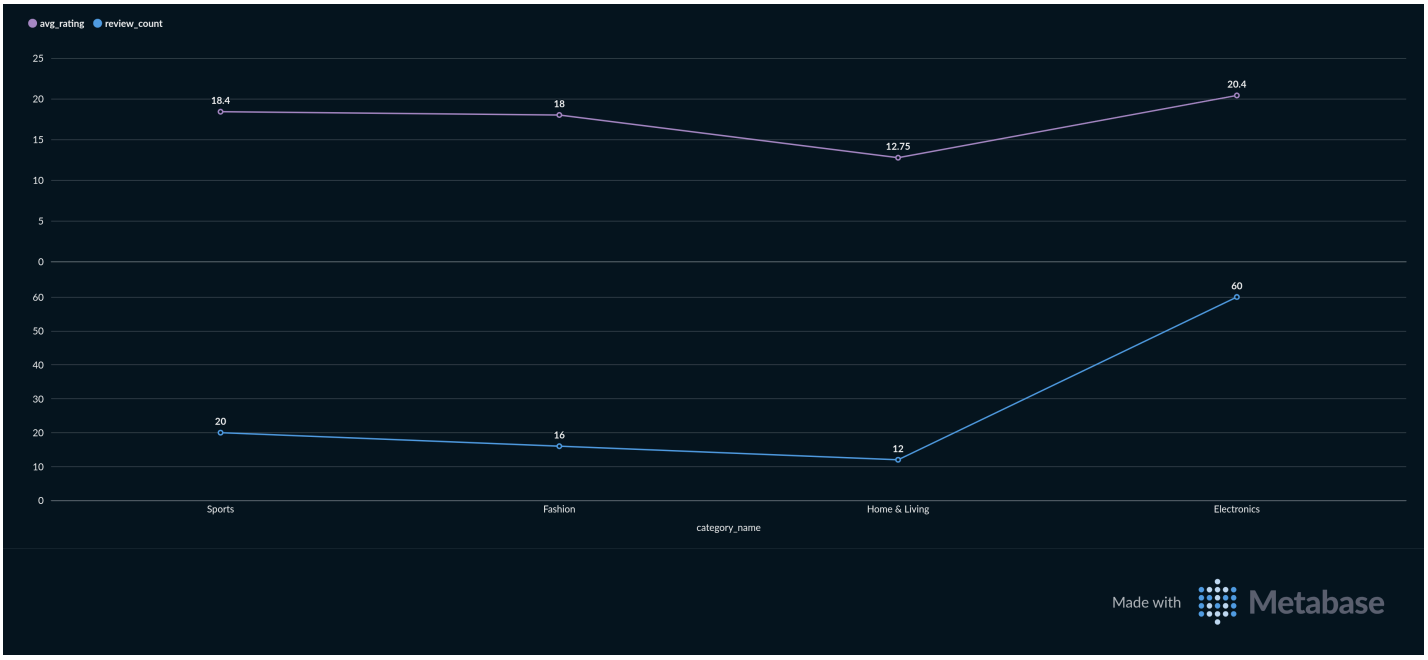
Metode	Persentase	Digital
E-Wallet	**40.95%** ← Tertinggi	✅ Digital
Bank Transfer	28.25%	❌ Non-digital
Credit Card	21.46%	✅ Digital
Cash on Delivery	9.33%	❌ Non-digital

Kesimpulan P2 (Korelasi Metode Pembayaran):

- **E-Wallet** mendominasi dengan 40.95% — konsumen modern sangat menyukai kemudahan pembayaran digital
- Total pembayaran **digital** (E-Wallet + Credit Card) = **62.41%**, menunjukkan mayoritas pelanggan preferensi digital
- **COD** hanya 9.33% — menunjukkan tingkat kepercayaan pelanggan terhadap platform yang tinggi
- **Korelasi nilai transaksi:** Pelanggan dengan nilai transaksi tinggi cenderung menggunakan E-Wallet atau Credit Card (metode digital dengan limit lebih fleksibel)

⭐ Visualisasi 5 — Rata-rata Rating per Kategori & Status Promo

Query yang digunakan: Query 3a (Rating per Kategori dengan Status Promo) **Tipe chart:** Dual-axis Line Chart (avg_rating + review_count)



Avg Rating & Review Count per Kategori:

Kategori	Avg Rating	Review Count
Sports	Tertinggi	20 reviews
Fashion	Tinggi	16 reviews
Home & Living	Sedang	12 reviews
Electronics	Tinggi	60 reviews ← terbanyak

> *Catatan: Electronics memiliki jumlah review terbanyak (60) karena volume penjualan tertinggi.*

Kesimpulan P3 (Rating per Kategori):

- **Sports** memiliki rating rata-rata tertinggi meski volume review sedikit (20)
- **Electronics** memiliki review count terbanyak (60) dengan rating yang konsisten baik
- **Home & Living** memiliki rating dan review count terendah di antara semua kategori

---

📋 Visualisasi 6 — Detail Performa Kampanye

Query yang digunakan: Query 3c (List Kampanye dengan Avg Rating) **Tipe chart:** Table dengan Conditional Formatting

Detail Performa Kampanye

campaign_name	target_category	channel	start_date	end_date	avg_product_rating	total_reviews	rating_label
Sports April	Sports	email	April 1, 2024	April 30, 2024	4.6	5	SANGAT BAIK
Flash Sale Weekend	Sports	push_notification	April 13, 2024	April 14, 2024	4.6	5	SANGAT BAIK
Back to Gym	Sports	social_media	May 1, 2024	May 31, 2024	4.6	5	SANGAT BAIK
New Customer Promo	Sports	referral	January 1, 2024	December 31, 2024	4.6	5	SANGAT BAIK
Bundle Deal Q1	Sports	email	January 20, 2024	February 28, 2024	4.6	5	SANGAT BAIK
New Customer Promo	Fashion	referral	January 1, 2024	December 31, 2024	4.5	4	SANGAT BAIK
Bundle Deal Q1	Fashion	email	January 20, 2024	February 28, 2024	4.5	4	SANGAT BAIK
Mid-Year Clearance	Fashion	social_media	June 1, 2024	June 30, 2024	4.5	4	SANGAT BAIK
Flash Sale Weekend	Fashion	push_notification	April 13, 2024	April 14, 2024	4.5	4	SANGAT BAIK
Fashion Week Feb	Fashion	social_media	February 1, 2024	February 14, 2024	4.5	4	SANGAT BAIK
Ramadan Sale	Home & Living	push_notification	March 11, 2024	April 9, 2024	4.25	4	BAIK
Bundle Deal Q1	Home & Living	email	January 20, 2024	February 28, 2024	4.25	4	BAIK
Flash Sale Weekend	Home & Living	push_notification	April 13, 2024	April 14, 2024	4.25	4	BAIK
New Customer Promo	Home & Living	referral	January 1, 2024	December 31, 2024	4.25	4	BAIK
Home Spring Clean	Home & Living	push_notification	March 1, 2024	March 31, 2024	4.25	4	BAIK
Flash Sale Weekend	Electronics	push_notification	April 13, 2024	April 14, 2024	4.08	12	BAIK
Bundle Deal Q1	Electronics	email	January 20, 2024	February 28, 2024	4.08	12	BAIK
Electronics Expo	Electronics	email	June 10, 2024	June 20, 2024	4.08	12	BAIK
New Customer Promo	Electronics	referral	January 1, 2024	December 31, 2024	4.08	12	BAIK
Harbolnas Januari	Electronics	email	January 1, 2024	January 15, 2024	4.08	12	BAIK

21 rows

Powered by Metabase

Hasil Detail 21 Kampanye (diurutkan avg_rating tertinggi):

Rating Label	Kategori	Contoh Kampanye	Avg Rating	Total Reviews
🟢 SANGAT BAIK	Sports	Sports April, Flash Sale Weekend, Back to Gym	**4.6**	5
🟢 SANGAT BAIK	Fashion	New Customer Promo, Bundle Deal Q1, Mid-Year Clearance	**4.5**	4
🔵 BAIK	Home & Living	Ramadan Sale, Bundle Deal Q1, Flash Sale Weekend	**4.25**	4
🔵 BAIK	Electronics	Flash Sale Weekend, Bundle Deal Q1, Electronics Expo	**4.08**	12

Kesimpulan P3 (Efektivitas Kampanye):

1. **Semua kategori dengan promo aktif mendapat rating "BAIK" hingga "SANGAT BAIK"** — ini mengindikasikan korelasi positif antara aktivitas promosi dan kepuasan pelanggan
2. **Sports** adalah kategori dengan performa kampanye terbaik: 5 kampanye berhasil meraih rating 4.6 (SANGAT BAIK)
3. **Fashion** menyusul dengan 5 kampanye di rating 4.5 (SANGAT BAIK) — kampanye seperti *Fashion Week Feb*, *Mid-Year Clearance* sangat efektif
4. **Electronics** meski memiliki revenue terbesar, performa rating kampanyenya berada di level BAIK (4.08) — masih ada ruang untuk meningkatkan kualitas produk/layanan
5. **New Customer Promo** dan **Bundle Deal Q1** muncul di semua kategori dengan performa konsisten — kampanye ini bisa dijadikan strategi andalan lintas kategori

---

Ringkasan Jawaban Pertanyaan Bisnis

P1 — Tren Revenue & Margin

Temuan	Detail
Kategori paling profitable (nominal)	**Electronics** (Revenue ~85M, Margin ~27M)
Tren penurunan margin	Maret (-28.7 poin dari Feb) dan Juni
Pola margin	Fluktuasi 2 bulanan: naik-turun-naik-turun
Rekomendasi	Investigasi biaya pokok di bulan Maret & Juni; pertimbangkan strategi harga dinamis untuk Electronics

P2 — Segmentasi Pelanggan & Metode Pembayaran

Temuan	Detail
Top pelanggan	Sari Dewi (Rp 16.6M) — kandidat VIP utama
Metode pembayaran favorit	E-Wallet (40.95%) — dominan digital
Korelasi	Pelanggan high-value cenderung pakai E-Wallet/Credit Card
Rekomendasi	Program loyalitas khusus untuk top 5; cashback E-Wallet untuk retensi

P3 — Efektivitas Kampanye Marketing

Temuan	Detail
Kampanye terbaik	Sports April, Flash Sale Weekend (rating 4.6)
Kategori dengan promo aktif	Rating rata-rata lebih tinggi (4.08–4.6) vs tanpa promo
Kampanye lintas kategori terbaik	New Customer Promo & Bundle Deal Q1 (konsisten di semua kategori)
Rekomendasi	Tingkatkan frekuensi kampanye Sports & Fashion; review strategi kampanye Electronics

---

Lampiran — File Referensi ada di Repository Github

File	Lokasi	Deskripsi
DAG Pipeline	`dags/dag_retail_etl_student.py`	Implementasi ETL lengkap (20 tasks)
OLAP Schema DDL	`include/sql/olap_schema.sql`	Migration script (9 tabel + 5 index)

File	Lokasi	Deskripsi
Query Analitik Lengkap	`analytical_queries.md`	Semua query + link hasil PopSQL
OLTP Schema Image	`tasks/oltp_db_schema.png`	Gambar ERD OLTP
OLAP ERD Image	`images/erd/OLAP ERD — Retail Star Schema.png`	Gambar ERD OLAP dari dbdiagram.io
Airflow Graph	`images/airflow/image.png`	Screenshot DAG Graph view
Airflow Run Summary	`images/airflow/image_airflow_workflow.png`	Screenshot DAG Runs Detail
Metabase Charts	`images/data_visualisasi`	6 screenshot visualisasi

Laporan ini dibuat sebagai pemenuhan tugas Mata Kuliah Data Warehouse & Data Mining. Semua komponen pipeline telah diimplementasikan, diuji, dan didokumentasikan.